École d'ingénieur Denis Diderot
Université Paris Cité

THALES

Implementation of the language server protocol and a Visual Studio Code extension for C/ACSL programming language features.

June 14, 2024

Internship report

Djamila Mohamed
2023–2024

Supervisor:
Adel Djoudi, Formal Methods Expert

Contents

1 Introduction

1.1 Thales DIS, IBS

Thales Digital Identity and Security is the global business unit (GBU) of Thales which operates in the field of secure software, providing biometric and encryption solutions. Within the DIS GBU is the Identity and Biometric Solutions business line (BL) which is in charge of digital identity (e.g.: cards and passports). The IBS BL is divided in three subunits: ID Documents Solutions (IDDS), Digital ID & Verification Solutions and Public Security. I am assigned to the IDDS subunit as an intern software developer, under the supervision of Adel Djoudi, formal methods expert at Thales DIS.

1.2 Context

In IDDS, developers use program analyzers such as Frama-C[1], a C source file analyzer developed by CEA and Inria. This software provides a precise analysis of code and potential security flaws by expressing functional specifications through ANSI/ISO C Specification Language (ACSL)[2] annotations. But Frama-C currently does not provide a convenient integrated development environment (IDE) to edit and browse C and ACSL code.

The goal of this project is to develop a software solution to offer language programming features for C/ACSL in IDEs including code navigation, completion suggestions, and interaction with Frama-C code analysis.

2 Background

2.1 The Language Server Protocol

Developers working on large scale projects often need a code editing environment with the adequate tooling for the languages they use in their workspace. Multiple solutions such as Eclipse or IntelliJ IDEA exist for Java projects, as well as other popular programming languages. But the complexity of developing one tool for one language and one specific editor is tedious as it forces developers to implement the same language programming features for every code editor with each programming language. To overcome these, a recent solution was developed: the Language Server Protocol (LSP)[3]. My first task was to do bibliographic research about its use cases and some of its implementations to prepare the implementation of the project and understand its challenges.

The language Server Protocol (LSP) is built on top of JSON-RPC protocol[4], a remote procedure call protocol that uses JSON as data format.

2.2 Frama-C plugin development

We choose to implement the ACSL language server as a Frama-C plugin to leverage the parsing features of Frama-C for ACSL code. It was therefore necessary to read and follow the Frama-C (Nickel 28.0) plug-in development guide. The Frama-C plug-in will act as the server for the LSP and the VS Code extension as the client. The plug-in development is the main task of the project. VS Code was chosen since it is used by most developers especially in Thales DIS. As long as we have the server part, any editor can provide an extension to connect to the Frama-C server.

2.3 VS Code Extensions

The architecture of VS Code extensions is based on a client-server model, which aligns perfectly with the Language Server Protocol. VS Code provides an API with TypeScript libraries for developing different types of extensions, particularly, language extensions. A Language extension offers declarative (e.g.: syntax highlighting, bracket autoclosing) and programmatic (e.g.: hover information, jump to definition) language features.

3 Development

3.1 Server implementation

One of the primary tasks is to select the appropriate method for communicating between the VS Code client and the Language Server. In this case, the implementation requires leveraging the OCaml Unix module and using the TCP socket transport method to establish a connection between the two components. This communication channel enables the exchange of requests and responses.

[1] https://frama-c.com/html/overview.html
[2] https://frama-c.com/html/acsl.html
[3] https://microsoft.github.io/language-server-protocol/overviews/lsp/overview/
[4] https://www.jsonrpc.org/specification

To facilitate the communication between client and server, the extension needs to handle both receiving and sending requests using the Unix module in OCaml. The VS Code client creates a server process listening on 'localhost:8001', the Frama-C plug-in will connect to that address and receive requests from it. Port 8001 was chosen arbitrarily for development purposes.

When client and server are connected, they must agree on which features both can provide through the 'initialize' handshake, where the client first sends the features it can support and the server responds with its capabilities. The VS Code process is executed in a Node.js runtime environment.

Below is a sequence diagram representing the lifecycle of the VS Code application.

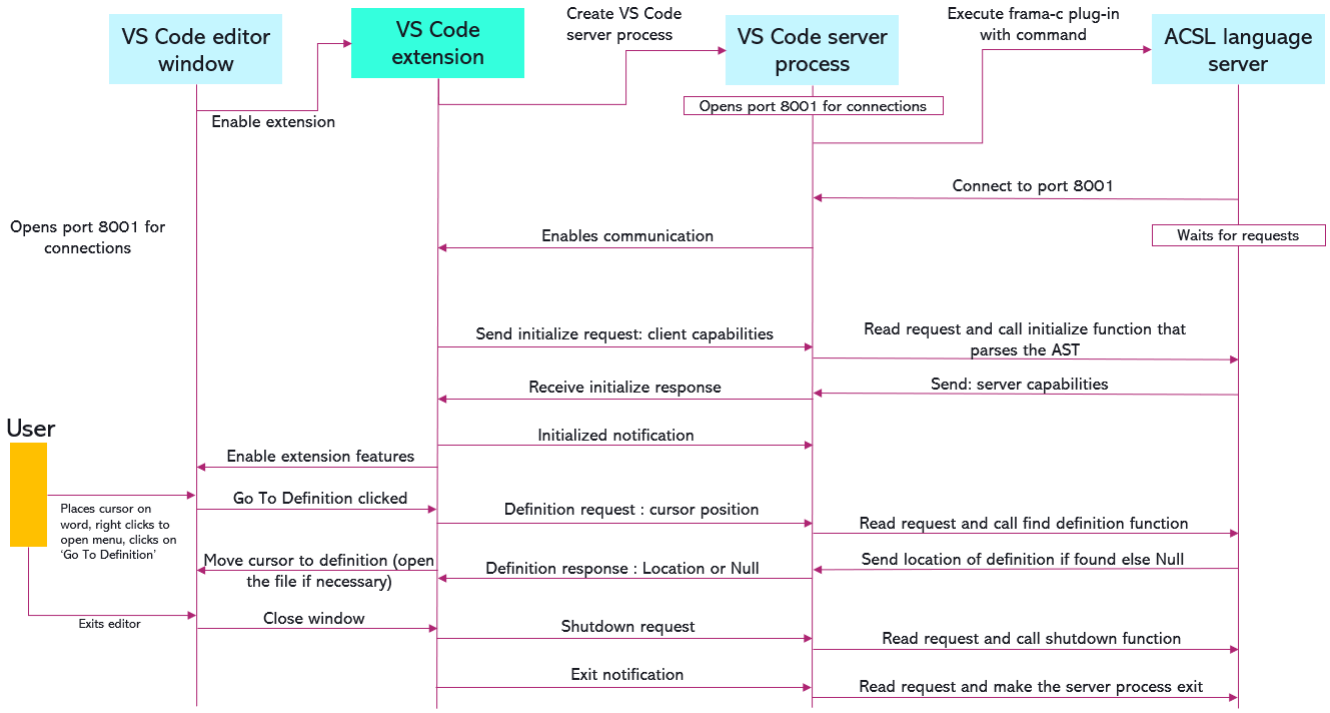

Figure 1: Lifecycle of the VS Code language extension for ACSL

3.2 Initializing the editor's workspace folder

When the user opens a folder with C source files, the language server initializes the workspace folder: in order to use the Frama-C module's functions, the plug-in converts the source code files into their C Intermediate Language (CIL) representation, which provides a detailed hierarchical structure of the source code. It captures the syntactic elements and relationships within the code, for deeper analysis and processing. This representation allows the server to efficiently navigate and manipulate the code, supporting features like 'Go To Definition' and other code navigation tools.

3.3 'Go To Definition' feature

The 'Go To Definition' feature of Visual Studio Code allows developers to easily navigate to the definition of symbols within their code. In this context, the challenges involve implementing a VS Code extension that supports this feature for two languages associated with the same file extension ('.c' and '.h').

When a user requests to find the definition of a symbol at a specific position in the code, the server first identifies the name of the symbol located at that position. This involves parsing, analyzing the file and extracting the relevant word based on the provided line and column numbers. The code then initiates a search through various sections of the code to find where the symbol is defined. This search involves visiting different parts of the source file's Abstract Syntax Tree (AST) and concerns both ACSL and C code:

- ACSL Definitions: The code checks if the symbol matches any logical predicates or terms, which are specific to tools like Frama-C used for static analysis of C programs.
- C Definitions: It examines global entities, such as enums, structures, functions, and variables, to see if the symbol matches any of these.

Each of these searches uses visitor patterns, which traverse and inspect nodes in the AST, comparing each node's name to the symbol's name.

If a definition is located, the exact file path and the range of lines and columns where the symbol is defined are packaged into a JSON response. If no definition is found, a response indicating a null result is returned. This response allows the LSP client (the user's text editor or IDE) to navigate directly to the symbol's definition or handle the case where no definition exists.

4 Ongoing tasks

- Create end-to-end tests for VS Code extension
- Create tests for Frama-C plug-in
- Implement 'Diagnostics' feature of LSP (compilation errors, warnings, etc.)
- Implement the display of CIL of source code on demand